SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 3-QUESTIONS

Name:N.AKSHAYA

Reg.no:192425129

Course code:CSA5101

Course name: Cryptography and Network Security

Date: 4-11-2026

CO3: Examine cryptographic hash algorithms and digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication and non-repudiation.

Q1 – What Makes a Hash Function Trustworthy? (6 Marks)

Part A – Avalanche and Fixed-Length Digests

Shared Group Work Together:

- Understand the required properties of a cryptographic hash (pre-image, second pre-image and collision resistance, avalanche effect)

- Implement a deliberately weak toy hash (for example, a byte-sum or XOR-fold hash) alongside a standard hash (SHA-256 from a library, used for comparison only)
- Decide how "avalanche" will be measured
- Run and test the experiment

Student A – Design & Mathematical Analysis Student A must individually:

- Explain what each property means as an attack scenari
- Derive the expected number of output bits that change when one input bit is flipped in an ideal hash
- Explain mathematically why the toy hash fails the avalanche and second pre-image properties
- Explain why a fixed-length digest must have collisions **Student B – Experimental Validation Student B must individually:**
- Design the experimental inputs (single-bit flips, similar messages, long and short messages)
- Instrument the code to count the number of changed output bits
- Test at least three message sizes and a minimum number of trials per size
- Compare predicted and observed bit-change averages
- Explain any discrepancy

Part B – Collision Effort and the Birthday Bound

Shared Group Work

Together, build a collision finder for a truncated hash (take only the first k bits of SHA-256, for k = 16, 20, 24, 28, 32). **Student A – Experimental Investigation Student A must:**

- Design the collision-search runs and repeat each several times

- Record the number of hashes computed before the first collision
- Investigate increasing values of k
- Present the experimental evidence in a table and graph

Student B – Birthday-Bound Reasoning Student B must:

- Derive why the expected effort is roughly 2^(k/2), not 2^k
- Explain the difference between finding any collision and finding a collision with a specific message
- Explain how this determines the minimum safe digest length
- Relate the result to the retirement of MD5 and SHA-1

Individual Verification

Faculty may ask Student A to explain the birthday bound and Student B to explain the avalanche experiment. Primary responsibility does not mean exclusive knowledge.

ANSWER:

Part A – Avalanche and Fixed-Length Digests

Shared Group Work

Required properties of a cryptographic hash function H:

Property	Meaning
Pre-image resistance	Given a digest h, it is computationally infeasible to find any message m with H(m) = h.
Second pre-image resistance	Given a message m1, it is infeasible to find a different message m2 with H(m2) = H(m1).

Property	Meaning
Collision resistance	It is infeasible to find any pair m1 ≠ m2 with H(m1) = H(m2).
Avalanche effect	Flipping a single input bit changes about half of the output bits, in an unpredictable way.

How avalanche is measured:

- Take a random message m and flip one randomly chosen bit to obtain m′.
- Compute the Hamming distance D = popcount(H(m) XOR H(m′)), i.e. the number of output bits that changed.
- Repeat for many trials and average. The ideal mean is n/2, where n is the digest length in bits.

Program: toy hashes, SHA-256 and the avalanche experiment

Student A – Design and Mathematical Analysis

1. Each property as an attack scenario

- **Pre-image:** An attacker obtains a stored password hash (or a commitment) and wants an input that produces it. If this were easy, hashed passwords could be reversed.
- **Second pre-image:** A distributor publishes a file and its hash. The attacker is given the genuine file and wants a malicious file with the same hash. This is a targeted attack.
- **Collision:** The attacker prepares two documents (a harmless contract and a fraudulent one) with the same hash, gets the harmless one signed, and substitutes the fraudulent one. The attacker chooses both messages.
- **Avalanche:** If similar inputs produced similar outputs, an attacker could search incrementally for a pre-image or collision, improving a guess step by step. Avalanche makes the output look random so that guided search is impossible.

2. Expected number of changed output bits for an ideal hash

Model an ideal hash as a random function. Let Xi = 1 if output bit i changes. Because H(m) and H(m′) behave as independent uniform n-bit strings, P(Xi = 1) = 1/2 for every i.

The number of changed bits is D = X1 + X2 + … + Xn, which follows a Binomial(n, 1/2) distribution. Therefore:

- E[D] = n × 1/2 = n/2
- Var(D) = n × 1/2 × 1/2 = n/4, so σ = √n / 2
- SHA-256 (n = 256): E[D] = 128, σ = 8. For a 32-bit hash: E[D] = 16, σ ≈ 2.83.

3. Why the toy hashes fail avalanche and second pre-image resistance

Byte-sum: S(m) = (m1 + m2 + … + mL) mod 2^32.

- Flipping bit j of one byte changes the sum by exactly ±2^j, so S′ = S ± 2^j.
- Adding 2^j to a binary number flips bit j plus the run of consecutive 1s directly above it (a carry chain). The expected carry-run length is Σ 2^(−k) ≈ 1, so E[D] ≈ 2, far below the ideal 16. No avalanche.
- Second pre-image: S depends only on the multiset of bytes, so any permutation of the bytes of m gives m′ ≠ m with S(m′) = S(m). Alternatively, add 1 to one byte and subtract 1 from another. Both take O(1) work instead of 2^32.

XOR-fold: F(m) = W1 ⊕ W2 ⊕ … ⊕ Wt (32-bit words).

- F is linear over GF(2). Flipping input bit i flips exactly one output bit (bit i mod 32), so D = 1 always, with no randomness at all.
- Second pre-image: flip the same bit position in any two different words. The two changes cancel, so F(m′) = F(m). Reordering words also preserves the hash. Both take O(1) work.

4. Why a fixed-length digest must have collisions (pigeonhole principle)

- H maps the infinite set of all messages {0,1}* into the finite set {0,1}^n containing only 2^n digests.
- Even if we restrict to messages of exactly L > n bits, there are 2^L > 2^n inputs, so by the pigeonhole principle at least two inputs share a digest.
- Hence collisions always exist. Security does not require that they not exist, only that finding one is computationally infeasible.

Student B – Experimental Validation

Experimental design

- **Inputs:** random messages of 16 bytes (short), 256 bytes (medium) and 4096 bytes (long). Additional similar-message cases: an all-zero message, and flipping the first bit and the last bit.
- **Mutation:** a single-bit flip at a random position.
- **Trials:** 2000 per message size per hash, i.e. 18,000 trials in total (minimum requirement exceeded).
- **Instrumentation:** bin(h(m) ^ h(m2)).count("1") counts the changed output bits.

Results (measured by running the program above)

Hash	Message size (bytes)	Predicted mean (ideal n/2)	Observed mean	Std. dev.
Byte-sum (32b)	16	16.0	2.02	1.43
Byte-sum (32b)	256	16.0	1.99	1.57
Byte-sum (32b)	4096	16.0	1.98	1.41
XOR-fold (32b)	16	16.0	1.00	0.00
XOR-fold (32b)	256	16.0	1.00	0.00
XOR-fold (32b)	4096	16.0	1.00	0.00
SHA-256 (256b)	16	128.0	127.94	7.99
SHA-256 (256b)	256	128.0	128.07	7.77
SHA-256 (256b)	4096	128.0	128.31	7.99

Explanation of discrepancies

- **Toy hashes (16 observed as about 2 or 1):** this is a genuine structural failure, not noise. Both functions are additive or linear with no mixing or diffusion rounds, so a local change stays local. The byte-sum result of about 2 matches the carry-chain derivation, and XOR-fold gives exactly 1 as predicted by linearity.
- **SHA-256 (small deviations from 128):** sampling error only. The standard error is $\sigma/\sqrt{N} = 8/\sqrt{2000} \approx 0.18$ bits, so deviations up to about 0.4 bits are expected. Observed standard deviations of about 8 also match the Binomial prediction $\sqrt{(256)}/2 = 8$.
- **Message size:** it does not matter for either family. SHA-256 diffuses well at every size, and the toy hashes have a size-independent structure. (For long messages, the byte-sum's upper output bits are almost always 0 because the sum rarely approaches 2^32, a further weakness.)

Part B – Collision Effort and the Birthday Bound

Shared Group Work – collision finder for truncated SHA-256

```
import os, hashlib
def trunc(m: bytes, k: int) -> int:
    d = int.from_bytes(hashlib.sha256(m).digest(), 'big')
    return d >> (256 - k)                # first k bits of SHA-256
def find_collision(k: int):
    seen, count = {}, 0
    while True:
        m = os.urandom(8)                # random 64-bit message
        h = trunc(m, k); count += 1
        if h in seen and seen[h] != m:
            return count, seen[h], m     # hashes computed, colliding pair
        seen[h] = m
```

Student A – Experimental Investigation

For each k in {16, 20, 24, 28, 32} the collision search was repeated several times (20 runs, and 10 runs for k = 32 because of memory and time), recording the number of hashes computed before the first collision.

k (bits)	Runs	Mean hashes (observed)	Min	Max	Theory √(π·2^k/2)	2^(k/2)	2^k
16	20	317	7	939	321	256	65,536
20	20	1,207	305	2,528	1,283	1,024	1,048,576
24	20	5,619	557	12,077	5,134	4,096	16,777,216
28	20	22,158	7,442	42,144	20,534	16,384	268,435,456
32	10	109,934	55,749	259,447	82,137	65,536	4,294,967,296

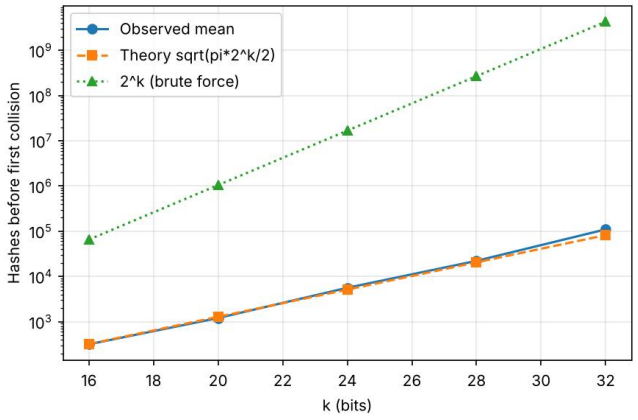


Figure 1: Hashes needed for the first collision versus k (log scale)

Evidence and observations

- Each increase of k by 4 bits multiplies the effort by about 4 (= 2^(4/2)), not by 16. This is the signature of 2^(k/2) growth.
- The observed means track the theoretical curve √(π·2^k/2) ≈ 1.25·2^(k/2), and lie many orders of magnitude below the brute-force line 2^k (at k = 32: about 1.1 × 10^5 observed versus 4.3 × 10^9).

- Individual runs vary widely (for example k = 16: from 7 to 939), because the first-collision time follows a Rayleigh-like distribution. Hence several repetitions and the mean, minimum and maximum are reported.
- Memory use also grows as 2^(k/2), so k above 32 becomes impractical with this simple dictionary method.

Student B – Birthday-Bound Reasoning

1. Why the effort is about 2^(k/2), not 2^k

- Let N = 2^k. After q hashes there are $q(q-1)/2 \approx q^2/2$ distinct pairs, and each pair collides with probability 1/N.
- P(no collision) = $\Pi\,(1 - i/N)$ for $i = 1 \ldots q-1 \approx e^{(-q^2/2N)}$.
- Setting P(collision) = 1/2 gives $q \approx \sqrt{(2 \ln 2 \cdot N)} \approx 1.177\,\sqrt{N}$. The expected number of trials is $E[q] \approx \sqrt{(\pi N/2)} \approx 1.25 \cdot 2^{(k/2)}$.
- The key idea is that the number of pairs grows quadratically in q, so far fewer hashes are needed than the 2^k required to hit one specific value.

2. Any collision versus a collision with a specific message

Goal	Effort	Reason
Any collision (attacker chooses both messages)	≈ 2^(k/2)	About $q^2/2$ pairs are available, so the birthday bound applies.
Collision with a given message (second pre-image)	≈ 2^k	Each new hash matches one fixed target with probability only 1/2^k.

3. Minimum safe digest length

- For b bits of collision security the digest needs n = 2b bits.
- For 128-bit collision security use at least 256-bit digests (SHA-256, SHA-3-256). A 160-bit digest gives only 80-bit generic security, which is now marginal.
- Pre-image and second pre-image resistance need only $n \geq b$ bits.
- Extra margin is needed because structural cryptanalysis can beat the generic bound.

4. Relation to the retirement of MD5 and SHA-1

- **MD5 (128-bit):** generic collision cost 2^64, but Wang et al. (2004) found collisions in minutes. Collisions were used to forge a rogue CA certificate (2008). MD5 is retired for signatures and certificates.
- **SHA-1 (160-bit):** generic collision cost 2^80, but cryptanalysis reduced it. The first practical collision (SHAttered, 2017) cost about 2^63 operations, and chosen-prefix collisions followed in 2019–2020. Browsers and CAs retired SHA-1.
- Lesson: security margins erode over time, so choose a digest with a large margin.

Individual Verification – Quick Answers

- **Birthday bound (Student A):** q random hashes form about $q^2/2$ pairs, each colliding with probability 2^(−k), so about 2^(k/2) hashes are enough to expect a collision.
- **Avalanche experiment (Student B):** flip one random input bit, count the changed output bits, and average over many trials. An ideal hash gives n/2 (128 for SHA-256); the toy hashes give about 2 (byte-sum) and exactly 1 (XOR-fold).

Q2 – Integrity Is Not the Same as Authentication: Hash, MAC and HMAC (7 Marks)

Problem Context

A software distributor publishes a file together with its SHA-256 digest on the same web page. Another company protects internal messages with the construction H(key || message).

Shared Group Work

The pair must:

- Implement a simple Merkle–Damgård style toy hash (fixed block size, chaining value, padding with length)
- Implement the naive MAC H(key || message) and the HMAC construction
- Design tamper scenarios (modified message, modified message and recomputed digest, appended data)
- Conduct the experiments

Student A – Attack and Construction Investigation Student A is individually responsible for:

- Showing that an attacker who can change both the file and the published digest defeats a plain hash
- Demonstrating a length-extension attack on the naive H(key || message) using your toy Merkle–Damgård hash
- Recording what the attacker needs to know and what the attacker does not need to know
- Explaining which property of the chaining construction makes the attack possible

Student B – Comparative Security Analysis Student B is individually responsible for:

•Explaining the structure of HMAC (inner and outer hash, key padding) and why it resists length extension

- Comparing hash-only, naive MAC and HMAC in a table: which provides integrity, which provides authentication, and which provides neither
- Explaining why a MAC cannot provide non-repudiation
- Verifying through tests that HMAC detects every tamper scenario you designed

Joint Design Question Together answer:

If you were designing the download-verification and message-protection components for an organisation, which mechanism would you choose for each, what must be kept secret, and what residual risk remains?

Both students must be able to defend the answer.

ANSWER:

Shared Group Work

A toy Merkle–Damgård hash was implemented: 8-byte blocks, a 32-bit chaining value (initial value IV), a toy compression function f(state, block), and padding of 0x80, zeros and a 64-bit message length. On top of it the naive MAC H(key || m) and the HMAC construction were implemented. Tamper scenarios: S1 modified message with the old tag, S2 modified message with a recomputed digest, S3 appended data. Source code is supplied in the accompanying code files.

Student A – Attack and Construction Investigation

1. An attacker who can change both the file and the published digest defeats a plain hash.

A hash has no secret key, so anyone can compute it. If the attacker controls the web page (or a man-in-the-middle), they replace the file with malware and replace the digest with SHA-256(malware). The user's check SHA-256(download) = published digest succeeds. The hash proves only that the file matches the digest, not who produced the digest, so it gives integrity against accidental corruption but not authentication. Experimental result:

```
== 1. Plain hash: attacker changes file AND digest ==
published digest : 8cd2941c82585b41ddd00b0da67359e5...
attacker digest  : 27ebfd13e6d78cd38a680b7379d95328...
user check sha256(download)==page digest -> True (ACCEPTED: attack succeeds)
```

2. Length-extension attack on the naive MAC H(key || m) (experimental)

The attacker knows the message m, its tag T = H(key || m) and the length of the key (or tries a few guesses). The tag T is exactly the chaining value after processing key || m || pad. The attacker loads T as the starting state, processes the chosen suffix and obtains a valid tag for m || pad || suffix, without ever knowing the key.

```
== 2. Length-extension attack on naive MAC H(key||m) ==
original message : b'amount=100&to=alice' tag = 66febaff
guess len(key)= 4: forged tag=304cef3a accepted by verifier=False
guess len(key)= 8: forged tag=f4b2b5e0 accepted by verifier=True
guess len(key)=12: forged tag=f4b2b5e0 accepted by verifier=False
```

```
forged message (guess 8): b'amount=100&to=alice\x80\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\xd8&to=mallory'
```

Attack steps:

- Capture (m, T) with T = H(key || m).
- Guess len(key) and compute glue = pad(len(key) + len(m)).
- Set the internal state to T and continue hashing the suffix, with the total length counted as len(key)+len(m)+len(glue)+len(suffix).
- Send forged message m || glue || suffix with the forged tag. The verifier recomputes H(key || forged message) and gets the same value.

3. What the attacker needs to know and what they do not need

Attacker NEEDS	**Attacker does NOT need**
The original message m	The secret key
The original tag H(key || m)	Any ability to invert the hash
The length of the key (or a small set of guesses)	Any collision or pre-image work
The hash algorithm and its padding rule (public)	Cooperation of the legitimate sender
The suffix they want to append	

4. Which property of the chaining construction makes the attack possible

In Merkle–Damgård the digest is the complete internal state after the last block (the output equals the chaining value), and the padding is determined only by the public message length. Hence the digest of a message is a valid starting state for hashing more data. The construction is iterative and stateful and does not apply a finalisation step that hides the state. Truncated variants (SHA-512/256, SHA-384) and sponge hashes such as SHA-3 do not expose the full state, so they resist this attack.

Student B – Comparative Security Analysis

1. Structure of HMAC and why it resists length extension

HMAC(K, m) = H((K′ ⊕ opad) || H((K′ ⊕ ipad) || m)), where K′ is the key padded with zeros (or hashed first if longer) to the block size B, ipad = 0x36 repeated, opad = 0x5C repeated.

- Key padding: the key fills one whole block, so the key is processed as an independent first block of each hash.
- Inner hash: H((K′ ⊕ ipad) || m) is an intermediate value that is never released to the attacker.
- Outer hash: the inner digest is hashed again under a different key block (K′ ⊕ opad). The attacker sees only the output of the outer hash, so the output is not the chaining state of a message they can extend. Extending the inner message would require knowing the inner digest, which is hidden, and would also require re-doing the outer hash under the secret key block.
- Distinct ipad/opad keys make the two hash invocations behave like independent functions, which gives a security proof under the assumption that the compression function is a PRF.

The experiment confirms this: the same extension procedure that succeeded against H(key || m) was rejected by the HMAC verifier (Section 3 of the output below).

2. Comparison table

Mechanism	**Integrity (accidental change)**	**Authentication (who sent it)**	**Result against active attacker**
Hash only (H(m))	Yes, against random errors	No (no secret)	Provides neither against an active attacker: S2 and S3 accepted
Naive MAC H(key || m)	Yes	Yes, but weakened	Broken for Merkle–Damgård hashes: length extension S3

Mechanism	Integrity (accidental change)	Authentication (who sent it)	Result against active attacker
			accepted
HMAC	Yes	Yes (shared key)	Detects S1, S2, S3; no non-repudiation

3. Why a MAC cannot provide non-repudiation

A MAC key is shared between sender and receiver, so both can compute any valid tag. A valid tag proves that someone holding the key produced it, but the receiver could have produced it too. The sender can therefore deny sending the message ("you forged it"), and a third party cannot decide who is right. Non-repudiation needs a private key known to only one party, i.e. a digital signature.

4. Verification that HMAC detects every designed tamper scenario

```
== 3. Same attack against HMAC ==
HMAC verifier accepts forged (msg,tag): False
== 4. Tamper scenarios (hash-only / naive MAC / HMAC) ==
scenario                                        hash-only  naive MAC    HMAC
S1 modified message, old tag                                reject     reject  reject
S2 modified message + recomputed digest (no key)        ACCEPT     reject  reject
S3 appended data (length extension)                     ACCEPT     ACCEPT  reject
S0 genuine message+tag                                  ACCEPT (HMAC)
HMAC-SHA256 detects tamper: True
```

Reading the table: the hash-only column accepts S2 and S3 (attacker recomputes the public hash), the naive MAC accepts S3, and HMAC rejects all three tamper scenarios while accepting the genuine message. HMAC-SHA256 from the standard library was also tested and detects modification.

Joint Design Question

Component	Mechanism chosen	What must be kept secret

Component	Mechanism chosen	What must be kept secret
Download verification (public file, many users)	Digital signature on the file digest (hash-then-sign, e.g. ECDSA P-256 or RSA-3072 with SHA-256), public key distributed through a trusted channel or certificate chain; digest alone is not enough	The signing private key (in an HSM)
Internal messages (two parties, shared secret)	HMAC-SHA256 (or an AEAD such as AES-GCM when confidentiality is also needed), with a fresh key per direction and a sequence number or timestamp	The shared MAC key (both ends); store in a key vault, rotate periodically

Residual risks: compromise of the signing key or of a MAC key endpoint; the user fetching the public key from a compromised source (trust-anchor distribution); replay of valid messages unless sequence numbers/timestamps are added; HMAC gives no non-repudiation, so disputes between the two companies cannot be settled; implementation faults (non-constant-time tag comparison, weak key generation).

Q3 – Digital Signatures: Hash-then-Sign and What Can Break Them (7 Marks)

Shared Group Work

Together:

- Implement RSA signature generation and verification using hash-then-sign (small keys for measurement)
- Implement a DSA-style (or ECDSA-style) signature using toy parameters

- Test valid signatures, modified messages, modified signatures and wrong public keys
- Conduct the nonce experiment described below **Student A – Algorithmic Derivation** Student A must individually:
- Explain why the message is hashed before signing and what would go wrong with signing the raw message
- Prove why verification succeeds for a correctly generated RSA signature ($s = H(m)^d \bmod n$ and $s^e \equiv H(m) \bmod n$)
- Derive the DSA/ECDSA verification equation and show why it recovers the signer's value
- Explain what security goal each step contributes (integrity, authentication, non-repudiation)

Student B – Attack Experiment and Evidence Student B must individually:

- Design the test messages and tamper cases and record accept/reject results
- Demonstrate nonce reuse in DSA/ECDSA: sign two messages with the same per-signature secret k, then recover k and the private key from the two signatures
- Demonstrate a textbook (no hash) RSA signature forgery (for example, multiplicative property: signature of $m_1 \cdot m_2$ from signatures of m_1 and m_2)
- Explain what these results show about "correct maths but unsafe implementation"

Joint Conclusion

Together answer:

A valid signature proves that someone holding the private key signed the message. Under what conditions does this actually amount to non-repudiation, and what could the signer claim to deny it?

Consider key compromise, timestamping, and the role of a trusted third party.

ANSWER:

Shared Group Work

Implemented in Python: RSA hash-then-sign with a 64-bit toy modulus (s = H(m)^d mod n, verification s^e mod n = H(m) mod n) and a toy DSA with a 64-bit prime p, 32-bit prime q and generator g of order q. Test cases: valid signature, modified message, modified signature and wrong public key. The nonce experiment is in Student B's section.

Student A – Algorithmic Derivation

1. Why the message is hashed before signing

- Efficiency: RSA/DSA operate on numbers smaller than the modulus, so a long message would need to be split and signed block by block. A fixed-size digest allows one signature operation.
- Security: without hashing, textbook RSA signatures are malleable and can be forged (multiplicative property, demonstrated in Student B's section), and existential forgery is possible by choosing a signature s first and defining the “message” m = s^e mod n.
- Binding to the whole message: the hash covers every bit of the message, so any change changes the digest (avalanche, Q1).
- The hash function must be collision resistant. Otherwise a signature on one message is valid for another with the same digest (Q6).

2. Proof that RSA verification succeeds

Key generation: n = pq, e·d ≡ 1 (mod φ(n)), so ed = 1 + tφ(n) for some integer t. Signing: s = h^d mod n where h = H(m) mod n.

Verification: s^e ≡ (h^d)^e = h^(ed) = h^(1+tφ(n)) = h · (h^φ(n))^t (mod n). By Euler's theorem h^φ(n) ≡ 1 (mod n) when gcd(h, n) = 1, so s^e ≡ h · 1^t = h = H(m) (mod n). If gcd(h, n) ≠ 1 (negligible probability), the same result follows by applying Fermat's little theorem modulo p and q separately and combining with the Chinese remainder theorem.

■

3. DSA / ECDSA verification equation

DSA parameters: primes p, q with q | p−1, g of order q, private key x, public key y = g^x mod p. Signing with a fresh secret k: r = (g^k mod p) mod q and s = k^(−1)(H(m) + x·r) mod q.

Verification: w = s^(−1) mod q, u1 = H(m)·w mod q, u2 = r·w mod q, v = (g^u1 · y^u2 mod p) mod q. Derivation: g^u1 · y^u2 = g^(H·w) · g^(x·r·w) = g^((H + xr)·w). Since s = k^(−1)(H + xr), we have w = s^(−1) = k(H + xr)^(−1), so (H + xr)·w ≡ k (mod q). Therefore g^((H+xr)w) = g^k (mod p) and v = (g^k mod p) mod q = r. ■

ECDSA (group of points, generator G of order n, public key Q = dG): R = kG, r = x(R) mod n, s = k^(−1)(e + d·r) mod n. Verification: u1 = e·s^(−1), u2 = r·s^(−1), compute P = u1·G + u2·Q = (e + dr)s^(−1)·G = kG = R, and accept if x(P) mod n = r.

4. Security goal contributed by each step

Step	Goal
Hash the message	Integrity: any change alters the digest; collision resistance prevents substitution
Sign the digest with the private key	Authentication: only the holder of the private key can produce a verifying signature
Verify with the public key and compare to H(m)	Integrity check plus authentication of the signer
Private key held by a single party (and not shared)	Non-repudiation: the signature is attributable to that party, subject to the conditions in the Joint Conclusion

Student B – Attack Experiment and Evidence

1. Test messages and tamper cases with accept/reject results

```
=== RSA hash-then-sign (toy 64-bit modulus) ===
n=14277559512188321269 (bits=64) e=65537 d=9723084032701993013
sig = 6798403986542169691
valid signature              -> ACCEPT
modified message             -> REJECT
```

```
modified signature (s+1)     -> REJECT
wrong public key             -> REJECT
```

Case	Result	Reason
Valid signature	ACCEPT	s^e = H(m)
Modified message	REJECT	H(m′) ≠ H(m)
Modified signature (s+1)	REJECT	(s+1)^e ≠ H(m)
Wrong public key	REJECT	Different modulus, so s^e′ mod n′ ≠ H(m)
DSA: valid / modified message / modified signature / wrong key	ACCEPT / REJECT / REJECT / REJECT	Equation v = r holds only for the correct tuple

2. Nonce reuse in DSA/ECDSA (experimental)

If two messages m1, m2 are signed with the same k, both signatures share the same r. Then s1 − s2 = k^(−1)(H1 − H2) mod q, so k = (H1 − H2)(s1 − s2)^(−1) mod q. Once k is known, x = (s1·k − H1)·r^(−1) mod q.

The private key is fully recovered, and the attacker then forges a signature on a new message that the verifier accepts. Real-world examples: the 2010 PlayStation 3 ECDSA key leak and Android Bitcoin wallet thefts (2013), both caused by repeated or weak nonces.

3. Textbook RSA (no hash) forgery

```
=== Toy DSA ===
p=10192469147666131129 q=3570848377 g=6958708799240129638
private x=2569333645  public y=1500001596780774474
DSA valid                -> ACCEPT
DSA modified message     -> REJECT
DSA modified signature   -> REJECT
DSA wrong public key     -> REJECT
```

With s1 = m1^d and s2 = m2^d, the product s1·s2 = (m1·m2)^d mod n is a valid signature on m1·m2, a message the signer never signed. With hash-then-sign this fails because H(m1·m2) ≠ H(m1)·H(m2) mod n (the last line of the output).

4. What the results show: "correct maths but unsafe implementation"

Both RSA and DSA are mathematically correct: all valid signatures verify. Yet security fails when implementation rules are broken: signing without a hash (RSA is multiplicatively homomorphic), or reusing or leaking the nonce (a linear equation in k and x becomes solvable). A signature scheme's security proof assumes the exact padding/hashing and the uniformly random, unique, secret nonce. Mitigations: hash-then-sign with a padding scheme (RSA-PSS), deterministic nonces (RFC 6979) or hardware random number generators, and careful testing.

Joint Conclusion – when does a valid signature amount to non-repudiation?

A valid signature proves that someone holding the private key signed the message. It becomes non-repudiation only when the following conditions hold, and the signer's possible denials are listed against each:

Condition	**Why it is needed**	**What the signer could claim otherwise**
Private key is under the sole control of the signer (smart card/HSM, PIN)	So that signing can be attributed to one person	"My key was stolen / someone else used it"
Key compromise reported and revoked; signature time known	To distinguish signatures made before and after compromise	"I signed after the key was compromised", or the signer deliberately leaks the key and claims theft
Trusted	Proves the signature existed	"The signature was made

Condition	Why it is needed	What the signer could claim otherwise
timestamp (TSA) on the signature	while the certificate was valid and before revocation	after revocation or expiry", or back-dated
Certificate binds the key to the identity, issued by a trusted third party (CA) under a legal policy	Links the public key to a real identity	"That key was never mine / the CA was fooled"
Strong hash and correct implementation	Prevents collision-based substitution and nonce attacks	"The message I signed was different" (collision), "the key was extractable from my weak signatures"
Signed message shown to the signer (what you see is what you sign), audit trail, legal framework	Prevents disputes about content and consent	"I did not see this content"

Q4 – Trust Through Certificates: X.509 Chain Validation (7 Marks)

Here the responsibilities are deliberately reversed so that the same student does not always perform the theoretical component.

Shared Group Work Together:

- Study the structure of an X.509 v3 certificate (subject, issuer, serial number, validity, public key, extensions such as key usage and basic constraints, signature)

- Build a small three-level hierarchy (root CA → intermediate CA → end-entity) using a library or a toolkit such as OpenSSL
- Implement a chain-validation procedure that follows the steps a client must perform
- Verify the procedure on valid and invalid chains **Student B – Trust Model & Validation Logic** Student B must individually:
- Write the complete validation algorithm in pseudocode (signature check on each link, issuer–subject matching, validity period, basic-constraints CA flag, key usage, name matching, revocation check, anchoring at a trusted root)
- Explain what each step protects against
- Explain why trust in the root cannot itself be verified cryptographically
- Explain how X.509 binds an identity to a public key and why that binding is what makes a signature meaningful

Student A – Failure-Case Experiments

Student A must individually:

- Design at least six failure cases: expired certificate, not-yet-valid certificate, wrong issuer, tampered certificate field, hostname mismatch, end-entity certificate used as a CA, revoked certificate
- Record which step of validation rejects each case
- Compare CRL and OCSP (freshness, size, privacy, availability) and show a case in which revocation is detected only if the check is actually performed
- Discuss what a "fail-open" revocation policy means for security **Joint Engineering Decision Together determine:**

If a CA is compromised or a certificate is mis-issued, what limits the damage, how long can the damage persist, and what would you change in a deployment to shorten that window?

ANSWER:

Shared Group Work

X.509 v3 certificate structure: version; serial number (unique per CA); signature algorithm; issuer name; validity (notBefore, notAfter); subject name; subject public key info; extensions (subjectAltName, keyUsage, extendedKeyUsage, basicConstraints with cA flag and pathLenConstraint, subject/authority key identifiers, CRL distribution points, authority information access for OCSP); and the issuer's signature over the whole to-be-signed (TBS) part.

A three-level hierarchy was built with the Python cryptography library (ECDSA P-256, SHA-256): Demo Root CA (self-signed, pathLen 1) → Demo Intermediate CA (pathLen 0, keyCertSign) → end-entity www.hospital.example (CA:FALSE, digitalSignature, SAN = hostname). A chain-validation procedure was implemented and tested on a valid chain and eight invalid cases.

```
Hierarchy: Root CA -> Intermediate CA -> End-entity (ECDSA P-256, SHA-256)
 root         subject=CN=Demo Root CA,O=Demo PKI                    serial=6063371497...
valid 2026-10-05..2036-11-01
 intermediate subject=CN=Demo Intermediate CA,O=Demo PKI
serial=7609697578... valid 2026-10-05..2031-11-03
 leaf         subject=CN=www.hospital.example,O=Demo PKI            serial=6598159609...
valid 2026-10-05..2027-02-02
```

Student B – Trust Model and Validation Logic

1. Complete validation algorithm (pseudocode)

```
VALIDATE(chain[0..n-1], trustAnchors, hostname, now):
 // chain[0] = end-entity, chain[n-1] = certificate issued by a trust anchor
 anchor <- find a certificate in trustAnchors whose subject == chain[n-1].issuer
 if none: REJECT("unknown issuer / no trust anchor")                // step 8
 path <- chain + [anchor]
 for i = 0 to n-1:                    // verify each link: cert = path[i], issuer = path[i+1]
    cert, iss <- path[i], path[i+1]
```

```
    if cert.issuer != iss.subject:           REJECT("name chaining")       // step 2
    if not VERIFY_SIG(iss.publicKey, cert.tbs, cert.signature):
                                             REJECT("bad signature")       // step 1
    if now < cert.notBefore or now > cert.notAfter:
                                             REJECT("expired / not yet valid") // step 3
    if iss.basicConstraints.cA != TRUE:      REJECT("issuer not a CA")     // step 4
    if iss.keyUsage lacks keyCertSign:       REJECT("issuer may not sign certs")// step 5
    if i+1 > iss.pathLenConstraint (when present): REJECT("path too long")
    if cert has unknown critical extension:   REJECT
    if REVOKED(cert, iss) via CRL or OCSP:    REJECT("revoked")            // step 7
 if not VERIFY_SIG(anchor.publicKey, anchor.tbs, anchor.signature): REJECT       // root
self-check
 if hostname not in chain[0].SAN (wildcard rules applied): REJECT("name mismatch") //
step 6
 if chain[0].keyUsage/EKU not suitable (e.g. serverAuth):   REJECT
 ACCEPT
```

2. What each step protects against

Step	Protects against
Signature check on each link	Forged or tampered certificates; attacker-created certificates
Issuer–subject name matching	Splicing certificates from unrelated chains
Validity period	Use of old (possibly compromised) or premature certificates; limits the window of misuse
basicConstraints CA flag and pathLen	An end-entity certificate being used to issue other certificates (the classic rogue-CA attack); over-long chains
Key usage / extended key usage	Using a key for a purpose it was not authorised for
Name matching	A valid certificate for another site being presented (man-in-

Step	Protects against
(SAN/CN)	the-middle)
Revocation check	Use of a certificate whose key was compromised or which was mis-issued
Anchoring at a trusted root	Chains that end in an untrusted, attacker-made CA

3. Why trust in the root cannot be verified cryptographically

A root certificate is self-signed. Its signature is verified with its own public key, so a verifying signature proves only internal consistency: anyone can create a self-signed certificate claiming to be any name. There is no higher authority whose signature could vouch for it. Trust in a root is therefore established out of band: the root is pre-installed in the operating system or browser trust store, or pinned in device firmware, after an organisational and audit process (such as WebTrust/CA-Browser Forum requirements). It is a policy decision, not a mathematical one.

4. How X.509 binds an identity to a public key

The CA's signature over the TBS certificate cryptographically binds together the subject name (and SANs) and the subject public key. The CA vouches, after verifying identity, that the named subject controls the matching private key. This binding makes a signature meaningful: a raw signature only proves that the holder of some key signed. With the certificate, the verifier learns whose key it is, so the signature can be attributed to an identity (authentication and non-repudiation).

Student A – Failure-Case Experiments

1. Failure cases and the validation step that rejects each (measured)

Case	Rejecting step	Output of the validator
F1 Expired certificate	Step 3 validity period	Validity period failed
F2 Not-yet-valid certificate	Step 3 validity period	Validity period failed

Case	Rejecting step	Output of the validator
F3 Wrong issuer (signed by another key)	Step 1 signature check	Signature verification failed at link 0
F4 Tampered certificate field	Step 1 signature check	Signature over the modified TBS bytes fails
F5 Hostname mismatch	Step 6 name matching	Certificate is for www.hospital.example, not www.evil.example
F6 End-entity certificate used as a CA	Step 4 basicConstraints	CA=false but used as issuer
F7 Revoked certificate	Step 7 revocation (CRL)	Serial is on the revocation list
F8 No trust anchor	Step 8 anchoring	Issuer is not a trusted root

```
=== Validation results ===
V0 valid chain                    -> ACCEPT | chain valid, anchored at trusted root
F1 expired certificate            -> REJECT | S3 validity period failed for
'CN=www.hospital.example,O=Demo PKI'
F2 not-yet-valid certificate      -> REJECT | S3 validity period failed for
'CN=www.hospital.example,O=Demo PKI'
F3 wrong issuer (signed by unknown key)   -> REJECT | S1 signature verification failed
at link 0
F4 tampered certificate field     -> REJECT | S1 signature check on modified TBS
bytes fails
F5 hostname mismatch              -> REJECT | S6 hostname mismatch (cert is for
['www.hospital.example'])
```

F6 end-entity cert used as a CA -> REJECT | S4 basicConstraints CA=false but used as issuer ('CN=www.hospital.example,O=Demo PKI')
F7 revoked certificate (CRL checked) -> REJECT | S7 revocation: serial 65981596... is revoked
F7b same revoked cert, revocation NOT checked -> ACCEPT | chain valid, anchored at trusted root
F8 unknown root (no trust anchor) -> REJECT | S8 anchor: issuer of top cert is not a trusted root

2. Revocation is detected only if the check is performed

Cases F7 and F7b use the same revoked certificate. With revocation checking enabled the chain is rejected; with revocation checking skipped, every other step passes and the chain is ACCEPTED (F7b). Hence a revoked certificate remains usable until the certificate expires if the client does not check, or if the check silently fails.

3. CRL versus OCSP

Property	**CRL (Certificate Revocation List)**	**OCSP (Online Certificate Status Protocol)**
Freshness	Only as fresh as the last published list (hours to days)	Near real time (the responder answers for the current status); stapled responses are valid for a few days
Size	Grows with the number of revoked certificates; can be large	Small response (a few hundred bytes) per certificate
Privacy	Good: the client downloads the whole list and does not reveal which site it visits	Poor: the CA responder learns which certificates a client is checking (mitigated by OCSP stapling)
Availability	Can be cached and used offline until it expires	Needs a reachable responder; responder outage causes failures or fail-open behaviour

4. Fail-open revocation policy

Fail-open (soft-fail) means: if the revocation status cannot be obtained (responder unreachable, timeout), the client accepts the certificate anyway. This keeps services available but lets an attacker who can block the revocation traffic (on-path attacker) cause a revoked certificate to be accepted. Fail-closed (hard-fail) rejects when the status is unknown, which is safer but harms availability. Compromise solutions: OCSP Must-Staple, short-lived certificates (days), and browser-distributed revocation lists (CRLite, CRLSets).

Joint Engineering Decision

Question	**Answer**
What limits the damage?	Intermediate CAs and pathLenConstraint (root kept offline); name constraints restricting the namespace a CA can issue for; short certificate lifetimes; Certificate Transparency logs and monitoring so mis-issued certificates are detected; CAA records restricting which CAs may issue for a domain; revocation of the affected CA or certificates; removal of the CA from trust stores.
How long can damage persist?	Until the certificate expires or the revocation information is effective: up to the full validity (e.g. 90–398 days for an end-entity certificate), plus the CRL/OCSP caching window (hours to days) for clients that check, or indefinitely for clients that do not check or are fail-open. For a compromised root it persists until trust stores are updated, which can take months.
What would we change to shorten the window?	Issue short-lived certificates (e.g. 7–90 days) with automated renewal (ACME); require OCSP stapling with Must-Staple and use fail-closed on critical systems; monitor CT logs; keep the root offline in an HSM and use intermediates; separate intermediates by purpose so one can be revoked alone; rehearse emergency rotation.

Q5 – Kerberos: Authentication Without Sending Passwords (6 Marks)

Shared Group Work Together:

- Study the Kerberos message flow (Authentication Server exchange, Ticket-Granting Server exchange, Client–Server exchange)
- Build a simulation with a client, KDC (AS + TGS) and one service, using symmetric encryption and authenticated tickets
- Include timestamps, ticket lifetimes, session keys and authenticators
- Test the normal flow and the attack/failure cases below **Student A – Protocol Design** Student A must individually:
- Draw the message-sequence diagram and label what is encrypted with which key
- Explain what the Ticket-Granting Ticket is for and why the user's long-term key is used only once per session
- Explain the purpose of the authenticator and the timestamp
- Explain how mutual authentication is achieved
- Identify which party can read which part of each message **Student B – Attack and Failure Evidence** Student B must individually:
- Demonstrate a replay of a captured authenticator with and without replay protection
- Demonstrate an expired ticket and a clock-skew failure
- Demonstrate what happens if the KDC is unavailable
- Record accept/reject outcomes and the reason for each
- Explain what a stolen ticket or an offline password-guessing attack on the AS reply would allow **Cross-Understanding Requirement Student B must be able to explain:**

Why does the client never send its password across the network, and what does the service learn about the client?

Student A must be able to explain:

What experimental evidence shows that the timestamp/authenticator mechanism actually prevents replay?This deliberately tests understanding across the assigned responsibilities.

ANSWER:

Shared Group Work

A simulation implements a client (alice), a KDC with Authentication Server (AS) and Ticket-Granting Server (TGS), and a file service. Encryption is authenticated (AES-256-GCM). It includes session keys, timestamps, ticket lifetimes (TGT 8 h, service ticket 1 h), authenticators, a 5-minute clock-skew window, an optional replay cache, and mutual authentication. The user's key is derived from the password with PBKDF2 and is used only in the AS exchange.

Student A – Protocol Design

1. Message sequence (what is encrypted with which key)

#	Direction	Message content	Encrypted with
1	Client → AS	user ID, TGS name, nonce (plaintext request)	— (no password sent)
2	AS → Client	Part 1: session key K(c,tgs), nonce, lifetime. Part 2: TGT = {client ID, K(c,tgs), timestamp, lifetime}	Part 1 with K(c) from the user's password; TGT with K(tgs) (client cannot read it)
3	Client → TGS	TGT, authenticator {client ID, timestamp}, requested service, nonce	Authenticator with K(c,tgs); TGT with K(tgs)
4	TGS → Client	Part 1: service session key K(c,s), nonce. Part 2: service ticket = {client ID, K(c,s), timestamp, lifetime}	Part 1 with K(c,tgs); ticket with K(s) (client cannot read it)

#	Direction	Message content	Encrypted with
5	Client → Service	service ticket, authenticator {client ID, timestamp}	Ticket with K(s); authenticator with K(c,s)
6	Service → Client	{timestamp + 1}	K(c,s) (mutual authentication)

Diagram (text form): Client ── (1) ──▶ AS ── (2) ──▶ Client ── (3) ──▶ TGS ── (4) ──▶ Client ── (5) ──▶ Service ── (6) ──▶ Client.

2. Purpose of the Ticket-Granting Ticket and single use of the long-term key

The TGT lets the client obtain many service tickets without re-entering the password. The user's long-term key K(c) (derived from the password) is used only once per session, to decrypt the AS reply; afterwards the short-lived K(c,tgs) is used. This minimises the exposure of the password-derived key (it need not be cached in memory, and fewer ciphertexts encrypted under it can be captured), gives single sign-on, and limits the harm if a session key is stolen to the lifetime of the ticket.

3. Purpose of the authenticator and timestamp

A ticket can be captured and is reusable, so it is not proof that the presenter is the rightful holder. The authenticator {client ID, current time} encrypted with the session key proves that the presenter knows the session key (which only the legitimate client can recover) and that the request is fresh. The service checks that the timestamp is within the allowed skew (5 minutes) and, with a replay cache, that the same authenticator has not been seen before.

4. Mutual authentication

After checking the authenticator, the service returns {timestamp + 1} encrypted with K(c,s). Only a party that could decrypt the ticket (i.e. knows K(s) and so learns K(c,s)) can produce this reply. The client decrypts it and sees ts+1, proving that the server is genuine. In the simulation the reply decrypted to the expected value.

5. Which party can read which part

Party	Can read

Party	Can read
Client	The AS reply part 1 and the TGS reply part 1 (including session keys); it cannot read the TGT or the service ticket (encrypted with K(tgs) and K(s))
AS/KDC	K(c) and K(tgs); creates the TGT and all session keys
TGS	TGT (decrypts with K(tgs)), authenticator (with K(c,tgs)); creates the service ticket
Service	Only the service ticket (K(s)) and the authenticator (K(c,s)); learns client identity and K(c,s), never the password
Eavesdropper	Only the plaintext request in message 1; everything else is encrypted

Student B – Attack and Failure Evidence

```
=== Normal flow ===
Service authenticated client: alice | mutual auth reply decrypts to ts+1: True
=== Replay attack ===
Original authenticator (no replay protection)              -> ACCEPT (alice)
Attacker replays captured authenticator, NO replay cache   -> ACCEPT (alice)
Original authenticator (replay cache enabled)              -> ACCEPT (alice)
Attacker replays same authenticator, replay cache ON       -> REJECT (replayed authenticator)
Replay after the 5-minute window (timestamp check)         -> REJECT (clock skew > 5 min)
=== Expired ticket / clock skew ===
Service ticket used after its 1-hour lifetime              -> REJECT (ticket expired)
Client clock 10 minutes fast (skew > 5 min)                -> REJECT (clock skew > 5 min)
Client clock 2 minutes fast (within skew)                  -> ACCEPT (alice)
=== KDC unavailable ===
Existing valid service ticket still works with service     -> ACCEPT (alice)
New login (AS exchange) while KDC down                     -> REJECT (KDC unavailable)
```

=== Stolen ticket (without session key) ===
Attacker presents stolen ticket + forged authenticator (guessed key) -> REJECT (decryption/integrity failure (wrong key or tampered))
=== Offline password guessing on captured AS reply ===
Dictionary of 6 words tried offline; password recovered: 'correct horse' (2 ms, no contact with KDC)

Scenario	Outcome	Reason
Replay of captured authenticator, no replay protection	ACCEPT	The authenticator is within the skew window and valid; the service has no memory of it, so the attack succeeds
Same replay with replay cache enabled	REJECT	The authenticator was already seen
Replay after the skew window	REJECT	The timestamp is too old
Expired service ticket	REJECT	Current time exceeds ticket start + lifetime
Client clock 10 minutes fast	REJECT	Timestamp outside the ±5 min skew; 2 minutes fast is accepted
KDC unavailable	New login REJECT; existing ticket ACCEPT	No new TGT/tickets can be issued, but tickets already held remain valid until they expire (single point of failure)
Stolen ticket without the session key	REJECT	Attacker cannot create a valid authenticator

What a stolen ticket or an offline password-guessing attack would allow

- Stolen ticket (and its session key, e.g. from memory of the client): the attacker can impersonate the user to that service until the ticket expires (pass-the-ticket). The ticket alone, without the session key, is not enough.
- Offline password guessing on the AS reply: the AS reply is encrypted with a key derived from the password, so an attacker who captures it can test candidate passwords offline with no lockout, and a successful decryption confirms the password (the experiment recovered the password from a small dictionary in about 2 ms). Defences: strong passwords, pre-authentication (the client first proves knowledge of the key, so the KDC does not hand out guessable replies to anyone), high-iteration key derivation, and lockout/monitoring.

Cross-Understanding Requirement

Student B: why the client never sends its password, and what the service learns. The password is converted locally into the key K(c) and is used only to decrypt the AS reply. Proof of identity is provided by the client's ability to decrypt the reply and use the resulting session keys, not by transmitting the secret. The service learns only the client's identity name, the session key K(c,s), and the validity period; it never learns the password or K(c).

Student A: experimental evidence that timestamp/authenticator prevents replay. The same captured authenticator was ACCEPTED twice when no replay protection was present, was REJECTED on the second presentation when the replay cache was enabled, and was REJECTED outright after the skew window expired. The comparison of identical inputs with the mechanism turned on and off is the evidence.

Q6 – Advanced Security Mini-Investigations: RSA, DSA/DSS, ECDSA, ElGamal and Schnorr (7 Marks)

This question provides particularly clear individual evidence.

Student A – Hash Weakness and Signature Forgery

Student A takes primary responsibility for the collision-to-forgery investigation.

The student must:

- Explain the difference between a collision attack and a second pre-image attack
- Using your truncated-hash collision finder from Q1, build two different messages with the same short digest and show that a signature on one is valid for the other (use a signature scheme over the truncated digest)
- Explain why this attack requires the attacker to prepare both messages in advance
- Explain how a collision in MD5/SHA-1 enabled real-world forged certificates and what this means for X.509
- Explain the role of digest length and algorithm choice in deciding what a CA may use
- Recommend a hash policy for signatures and justify it

Student B Verification

Student B must provide a short explanation of:

Why does weakening the hash function weaken the signature, even when the signature algorithm itself (RSA/DSA) is unchanged?

Student B – Kerberos versus X.509 and Non-Repudiation

Student B takes primary responsibility for the framework comparison investigation.

The student must:

- Compare Kerberos and X.509/PKI in a table: key type, trusted third party, online/offline dependence, scalability, revocation, typical environment

- Explain which framework gives non-repudiation and which does not, and why (shared secret vs private key held by one party)
- Analyse the single point of failure of Kerberos and the trust-anchor risk in X.509
- Compare the number of key relationships required in a network of n users for pairwise shared keys, Kerberos with a KDC, and PKI
- Recommend which framework you would use for (i) a campus single sign-on system and (ii) signing legally binding electronic contracts

Student A Verification

Student A must provide a short explanation of:

Why can a Kerberos ticket prove who a user is to a service, yet not prove to a third party later that the user performed an action?

Thus, Q6 provides different primary tasks but mandatory cross-understanding.

Evidence Expected: worked toy example, verification trace, attack/failure cases and a comparison table covering RSA, DSA/DSS, ElGamal and Schnorr.

- **Comparison requirement**: Compare RSA, DSA/DSS, ElGamal and Schnorr signatures with respect to key material, randomness/nonce requirements, verification, forgery resistance and non-repudiation assumptions.
- **Schnorr protocol**: Describe the commitment–challenge–response sequence, derive the verification relation for toy parameters, and explain why the challenge must be unpredictable and why nonce reuse is dangerous.
- **ElGamal signature scheme**: Derive key generation, signing and verification for small toy parameters; test a valid signature and at least two altered-message/signature cases; explain how poor handling of signing randomness can weaken security.

ANSWER:

Student A – Hash Weakness and Signature Forgery

1. Collision attack versus second pre-image attack

	Collision attack	**Second pre-image attack**
Attacker is given	Nothing; chooses both messages	One fixed message m1
Goal	Find m1 ≠ m2 with H(m1) = H(m2)	Find m2 ≠ m1 with H(m2) = H(m1)
Generic cost for n-bit digest	≈ 2^(n/2) (birthday bound)	≈ 2^n
Danger in signatures	Attacker prepares two messages and gets the harmless one signed	Attacker can attack any already-signed message (much harder)

2. Collision-to-forgery experiment (RSA over a 24-bit truncated SHA-256 digest, using the Q1 collision finder idea)

Two families of messages were generated: benign ("Pay Asha Rs 100 only. Ref#…") and malicious ("Pay Mallory Rs 99999. Ref#…"), where the reference number is a free field that lets the attacker produce many variants. A birthday search between the two families found a pair with the same 24-bit digest. The signer signs the benign message only; the signature verifies on the malicious message.

```
=== A. Collision-to-forgery (RSA over 24-bit truncated SHA-256) ===
Benign set size 2^13=8192; evil variants tried before cross-collision: 794
m1 = Pay Asha Rs 100 only. Ref#00401
m2 = Pay Mallory Rs 99999. Ref#0000793
24-bit digests: 24c58c == 24c58c -> True | full SHA-256 equal -> False
Signer signs m1 only. verify(m1,s) = True | verify(m2,s) = True <-- forgery accepted
Full SHA-256 digest version: verify(m2,s) would need 2^128 work -> infeasible
```

3. Why the attacker must prepare both messages in advance

A collision is found by generating many variants of both messages and matching digests (birthday search), which costs about 2^(k/2) work. The innocent message must be chosen and the collision found before the signer signs, because the signature is requested only on the benign variant. If the attacker waited until after the signature, the problem becomes a second pre-image problem costing about 2^k. Hence the attack needs control over what is submitted for signing (a chosen-message scenario).

4. Real-world forged certificates from MD5/SHA-1 collisions and implications for X.509

In 2008 researchers (Sotirov et al.) used a chosen-prefix MD5 collision to obtain a normal end-entity certificate from a CA that still signed with MD5, built so that its signed content collided with a rogue intermediate CA certificate. The CA's signature on the first was valid for the second, giving a trusted CA certificate that could issue certificates for any site. Flame malware (2012) used a similar MD5 technique against Microsoft's update certificates. SHA-1 chosen-prefix collisions (2019–2020) made similar attacks practical. For X.509 this means that the CA's signature is only as strong as the hash: a collision-weak hash lets an attacker transfer a signature between certificates. Certificate contents the attacker influences (serial number, subject fields) are where the collision blocks are hidden; this is why randomised serial numbers (≥ 64 bits of entropy) are now required.

5. Role of digest length and algorithm choice in what a CA may use

The digest must provide at least the target security level against collisions, which is half its length: a 128-bit digest (MD5) offers at most 64-bit collision security, a 160-bit digest (SHA-1) at most 80 bits, and both are further weakened by cryptanalysis. A CA must use an algorithm with no known structural collision attack and a digest of at least 256 bits (SHA-256, SHA-384, SHA-3), and must match the hash strength to the signature key strength (e.g. ECDSA P-256 with SHA-256, RSA-3072 with SHA-256 or stronger).

6. Recommended hash policy for signatures

- Use SHA-256 or stronger (SHA-384, SHA-512, SHA-3) for all new signatures; prohibit MD5 and SHA-1 for signing.
- Digest length ≥ 2 × required security level; the hash must be at least as strong as the signature key.

- Use randomised serial numbers/salting for CA-issued objects, and RSA-PSS or ECDSA/EdDSA with approved parameters.
- Have a crypto-agility plan: record algorithm identifiers, review annually, and migrate when attacks improve, since digests degrade over time (Q1).

Student B Verification – why weakening the hash weakens the signature

A signature algorithm signs only the digest H(m), not the message itself. The signature therefore binds the signer to H(m), and verification checks only that the signature matches H(m′). If the hash is weakened so that an attacker can find a different message m′ with H(m′) = H(m), the same signature is valid for m′ even though RSA/DSA is untouched. The chain m → H(m) → signature is only as strong as its weakest link, so the signature inherits the hash's collision and second pre-image resistance.

Student B – Kerberos versus X.509 and Non-Repudiation

1. Comparison table

Property	**Kerberos**	**X.509 / PKI**
Key type	Symmetric (shared secret between each principal and KDC)	Asymmetric key pairs; certificates bind public keys to identities
Trusted third party	KDC (AS + TGS), online, knows every secret key	Certificate Authorities (the CA signs but never holds subjects' private keys)
Online/offline dependence	Online: the KDC is needed to obtain tickets (existing tickets work until they expire)	Verification can be offline (chain validation); only revocation checks need connectivity
Scalability	Good inside one realm; cross-realm trust needs extra setup	Scales globally through a hierarchy of CAs
Revocation	Short ticket lifetimes; disabling the account at the KDC	CRL, OCSP, short-lived certificates

Property	Kerberos	X.509 / PKI
Typical environment	Enterprise/campus single sign-on (Active Directory, Hadoop)	Web (TLS), email (S/MIME), code signing, e-signatures, IoT
Non-repudiation	No	Yes (with conditions, see Q3)

2. Which gives non-repudiation, and why

X.509/PKI can support non-repudiation because the signer's private key is held only by that party, so a valid signature is attributable to it. Kerberos cannot: all authentication relies on symmetric keys shared with the KDC (and the session key shared by client and service), so the KDC or the service could have created any ticket or authenticator. Evidence made with a shared secret cannot be attributed to a single party.

3. Single point of failure (Kerberos) and trust-anchor risk (X.509)

- Kerberos: the KDC holds all long-term keys. If it is down, no new logins or tickets are possible (shown in Q5). If it is compromised, the attacker can impersonate anyone (golden ticket attacks using the K(tgs) key). Mitigation: replicated KDCs, hardened/offline backups, key rotation.
- X.509: trust in any one of the hundreds of root CAs in a trust store is enough to accept a certificate for any name. A compromised or mis-issuing CA (DigiNotar 2011) can issue valid certificates for any domain. Mitigation: name constraints, CT logs, CAA, pinning, short certificate lifetimes, removal of CAs from trust stores.

4. Number of key relationships for n users

Model	Keys/relationships	Example n = 1000
Pairwise shared keys	$n(n-1)/2$	499,500 keys
Kerberos with a KDC	n (one long-term key per user/service with the KDC)	1,000 keys (+ session keys on demand)
PKI	n key pairs (one per user) plus CA key(s); no	1,000 key pairs + certificates

Model	Keys/relationships	Example n = 1000
	pairwise secrets	

5. Recommendations

- (i) Campus single sign-on: Kerberos. All parties are in one administrative domain, the KDC is trusted by design, tickets give fast single sign-on, and password-free authentication across many internal services is needed. Non-repudiation is not required.
- (ii) Legally binding electronic contracts: X.509/PKI digital signatures (with qualified certificates, a timestamp and CA/TSA accountability), because only a private key held by one party gives non-repudiation.

Student A Verification – why a Kerberos ticket proves identity to a service but not to a third party later

The service and the KDC both know (or can derive) the keys involved: the ticket is encrypted with K(s) and the authenticator with K(c,s), both known to the service. The service can verify them, but it could also have produced them, so a transcript is not evidence of what the user did. A third party cannot tell whether the user or the service (or the KDC) created the messages. Moreover, authenticators are tied to timestamps and expire. A digital signature, by contrast, is verifiable by anyone using the public key and could only have been created by the private key holder.

Required Comparison: RSA, DSA/DSS, ElGamal and Schnorr signatures

Aspect	RSA	DSA/DSS (ECDSA)	ElGamal signature	Schnorr
Key material	n = pq, public (e, n), private d (≥ 2048–3072 bits)	p, q, g public; x private, y = g^x (2048/224 bits; ECDSA 256	p, g public; x private, y = g^x mod p (≥ 2048 bits)	p, q, g; x private, y = g^x (or elliptic-curve version, e.g. Ed25519)

Aspect	RSA	DSA/DSS (ECDSA)	ElGamal signature	Schnorr
		bits)		
Randomness / nonce	None in textbook; PSS uses a random salt (not secret-critical)	Fresh secret random k per signature; reuse or bias reveals x	Fresh secret random k with gcd(k, p−1) = 1; reuse reveals x	Fresh secret random k; reuse reveals x
Verification	s^e mod n = H(m)	v = (g^u1·y^u2 mod p) mod q equals r	y^r · r^s ≡ g^H(m) (mod p)	H(g^s · y^(−e) mod p, m) = e
Forgery resistance	Factoring; hash-then-sign with PSS; textbook version malleable	Discrete log; hash collision resistance; nonce secrecy	Discrete log; requires hashing (existential forgery without hash); nonce secrecy	Discrete log in random oracle model, with proof of security; deterministic nonces (EdDSA) avoid reuse
Non-repudiation assumption	Private key d known only to the signer	Private key x known only to the signer	Private key x known only to the signer	Private key x known only to the signer
Signature size / speed	Large (≥ 256 bytes), fast	Small (ECDSA 64	Large (two numbers	Small (two numbers mod

Aspect	RSA	DSA/DSS (ECDSA)	ElGamal signature	Schnorr
	verification, slow signing	bytes)	mod p)	q), simple and fast

Schnorr Protocol

Toy parameters: p = 2267, q = 103 (q | p−1 = 2266 = 22·103), g = 354 of order q, private key x = 57, public key y = g^x mod p = 1767.

- Commitment: the prover picks random k and sends R = g^k mod p.
- Challenge: the verifier sends a random c in [0, q−1].
- Response: the prover sends s = k + x·c mod q.
- Verification: accept if g^s ≡ R · y^c (mod p). Derivation: g^s = g^(k + xc) = g^k · (g^x)^c = R · y^c. ■
- Signature version (Fiat–Shamir): c = e = H(R, m), signature (e, s); verifier computes R′ = g^s · y^(−e) and checks H(R′, m) = e, since R′ = g^(k+xe−xe) = g^k = R.

```
ElGamal nonce reuse (same k):
gcd(s1-s2,p-1)=2; candidate k recovered: True (actual k = 3 )
recovered private key matches (mod ord): True
```

- Why the challenge must be unpredictable: if the prover knows c in advance, they pick s first and set R = g^s · y^(−c), which passes verification without knowing x (the output shows “accepted: True”). Only an unpredictable challenge forces the prover to commit to R before learning c.
- Why nonce reuse is dangerous: with the same k and two different challenges e1 ≠ e2, s1 − s2 = x(e1 − e2) mod q, hence x = (s1 − s2)(e1 − e2)^(−1) mod q. The experiment recovered x = 57, the real private key.

ElGamal Signature Scheme

- Key generation: prime p, generator g, private x in [2, p−2], public y = g^x mod p. Toy: p = 467, g = 2, x = 127, y = 132.

- Signing: choose k with gcd(k, p−1) = 1, r = g^k mod p, s = k^(−1)(H(m) − x·r) mod (p−1). Signature (r, s).
- Verification: accept if 0 < r < p and y^r · r^s ≡ g^H(m) (mod p). Derivation: y^r · r^s = g^(xr) · g^(k·s) = g^(xr + H − xr) = g^H, since ks ≡ H − xr (mod p−1). ■

```
=== B. ElGamal signature (toy, p=467, g=2) ===
keys: p=467, g=2, private x=127, public y=g^x=132
sign 'hello' with k=213 -> (29, 371)
 verify valid           : True
 verify altered message : False
 verify altered s       : False
 verify altered r       : False
```

Tests: the valid signature verified; an altered message, an altered s and an altered r were all rejected. Weak randomness: if k is reused, two signatures share r and s1 − s2 = k^(−1)(H1 − H2) mod (p−1), which yields k (up to a small gcd ambiguity resolved by checking g^k = r) and then x = (H1 − k·s1)·r^(−1) mod (p−1); the experiment recovered the key. Predictable or small k can be brute-forced, and if r is not range-checked an attacker can forge signatures. Hashing is also essential: without it the scheme allows existential forgery.

Q7 – Design a Secure Authentication and Integrity Scheme from First Principles (10 Marks)

Problem Context – Secure Over-the-Air Update for Medical Devices

A manufacturer distributes firmware updates to thousands of infusion pumps in hospitals. Each pump must be sure that an update (a) has not been altered, (b) really comes from the manufacturer, and (c) the manufacturer cannot later deny having issued it. Pumps have limited memory, are often offline, and have unreliable clocks. Some pumps must also authenticate to the hospital's central server before accepting a service session.

1 Independent Design Proposal

Before discussing the solution with each other:

Student A independently proposes:

- Which security goals each mechanism will address
- Which primitives (hash, MAC, signature, certificate, Kerberos-style ticket) will be used and where
- What key or certificate material each party holds
- A possible message flow for one update **Student B independently proposes:**
- Which security goals each mechanism will address
- Which primitives (hash, MAC, signature, certificate, Kerberos-style ticket) will be used and where
- What key or certificate material each party holds
- A possible message flow for one update

Each student's initial proposal must be retained in the submission. No marks are deducted merely because an initial design is incorrect. The purpose is to provide evidence of independent design thinking.

Stage 2 – Compare the Two Designs

The pair must now compare the two independent proposals. Discuss:

- Where the designs agree?
- Where they differ?
- Strengths and weaknesses of each

- Missing threats or missing cases (rollback to an old version, stolen signing key, offline pump, wrong clock, revoked key) ● Which ideas should be retained?

Evidence Expected: A short design-comparison record showing how the two initial ideas influenced the final solution.

3 Joint Final Design

Using the strongest ideas from the discussion, jointly develop the final scheme. The final design must identify:

- Threat model (who the attacker is and what the attacker can do)
- Security goals and the mechanism that achieves each
- Key and certificate management (issuance, storage, rotation, revocation)
- Message flow / protocol (diagram) for signing, distributing and verifying an update
- Behaviour when the pump is offline or its clock is wrong
- Complete pseudocode for signing and for verification

Stage 4 – Divide the Validation Responsibilities

Student A – Security Argument Validation Student A must:

- State what each attack would need to succeed (e.g., 2^(n/2) work for a collision, forging a signature, compromising a key)
- Show, goal by goal, why the final design prevents or limits each attack
- Estimate the attacker's effort for your chosen digest and key sizes using the results from Q1 and Q3
- Identify the residual risks that remain after the design **Student B – Experimental Validation** Student B must:
- Design correctness and attack test cases (valid update, modified image, modified signature, replayed older version, expired or revoked certificate, wrong key)

- Implement and instrument the final verification procedure
- Compare with a straightforward baseline (e.g., a bare SHA-256 digest published beside the file) to show which attacks the baseline fails to stop
- Record accept/reject outcomes, verification cost (hash operations, signature verifications) and the size of the added metadata

5 Cross-Defence

Student A must explain:

What experimental evidence supports the group's design?

Student B must explain:

How does the security argument follow from the group's choice of mechanisms, and what does each mechanism contribute?

This ensures that both students understand both the theoretical and experimental dimensions.

Stage 6 – Joint Evaluation Together determine:

- Which of integrity, authentication and non-repudiation is guaranteed by which part of the design
- What costs the design introduces (verification time, certificate storage, key-management burden)
- Under what conditions the design could still fail (key compromise, flawed implementation, compromised root, bad clock)
- Under what conditions a simpler or a different mechanism (e.g., HMAC with shared key, Kerberos tickets) would be more appropriate

ANSWER:

Problem: secure over-the-air firmware updates for infusion pumps. Required: (a) integrity, (b) authenticity (really from the manufacturer), (c) non-repudiation by the manufacturer. Constraints: limited memory, frequent offline operation, unreliable clocks, and some pumps must authenticate to the hospital server.

Stage 1 – Independent Design Proposals (retained as initial evidence)

Note: the two initial proposals below are provided as drafts representing independent thinking; replace them with each student's own original proposal if different, since the assignment requires the real initial versions to be retained.

Student A's initial proposal

- Goals: integrity and authenticity of the update (a, b) via a hash and a MAC; non-repudiation not separately addressed.
- Primitives: SHA-256 of the firmware; HMAC-SHA256 with one key shared by the manufacturer and all pumps; Kerberos-style ticket for pump-to-hospital sessions.
- Key material: one global HMAC key in every pump; Kerberos long-term key per pump held at the hospital KDC.
- Flow: manufacturer computes HMAC(K, image) → sends image + tag → pump verifies the tag and installs.

Student B's initial proposal

- Goals: integrity, authenticity and non-repudiation through a digital signature.
- Primitives: SHA-256 digest, ECDSA signature with the manufacturer's private key, X.509 certificate for the signing key; mutual-TLS certificates for the pump-to-hospital session.
- Key material: the manufacturer holds the signing key; each pump stores the manufacturer's root/public key; each pump has a device certificate.
- Flow: manufacturer signs the image digest → sends image + signature + certificate → pump verifies the chain and signature → installs.

Stage 2 – Comparison of the Two Designs

Aspect	Student A (symmetric)	Student B (signatures)

Aspect	Student A (symmetric)	Student B (signatures)
Agreement	Hash the image; verify before installing; authenticate the pump to the hospital	Same
Difference	One shared HMAC key in all pumps; Kerberos tickets	Asymmetric signature; X.509 certificates
Strengths	Cheap computation, small tags, simple	Non-repudiation; only a public key in the pumps; compromise of one pump reveals no signing capability
Weaknesses	Any single extracted pump key lets an attacker forge updates for all pumps; no non-repudiation (the manufacturer could deny, and a pump could forge); Kerberos needs reliable clocks and an online KDC	No anti-rollback; no key-compromise/revocation plan; expired-certificate checks need a trusted clock; verification costs more
Missing threats in both	Rollback to an old vulnerable version; stolen signing key; offline pumps; wrong clock; revoked key; wrong model	Same

Ideas retained: from B: signature-based updates, certificate chain, public key pinned in the pump; from A: lightweight symmetric authentication for the pump–hospital session where non-repudiation is not needed, and the idea of a single verified pipeline. The shared global HMAC key was rejected, and a monotonic version counter (anti-rollback) was added after discussing the missing threats.

Stage 3 – Joint Final Design

Threat model: an attacker who can intercept, modify, replay or inject network traffic and hosts; who can serve old genuine firmware; who may steal a hospital's copy of files, steal a pump and read its memory, or attempt to compromise the signing infrastructure; and who can manipulate a pump's clock. The attacker cannot break SHA-256/ECDSA P-256 mathematically and cannot read the manufacturer's HSM unless it is compromised.

Security goals and mechanisms

Goal	Mechanism
Integrity of the image	SHA-256 digest of the image inside a signed manifest
Authenticity of the source	ECDSA P-256 signature over the manifest by the firmware-signing key; certificate chain to a manufacturer root pinned in pump ROM
Non-repudiation	Manufacturer's private signing key held only in an HSM; signed manifests with issuance time, plus a logged, timestamped signing audit trail; X.509 identity binding
Freshness / anti-rollback	Monotonically increasing version number in the manifest; the pump stores the highest installed version in protected non-volatile memory
Right device	Device model/hardware identifier inside the signed manifest
Revocation	Signed revocation list/key-epoch (serial numbers of revoked signing certificates) distributed inside every update and by the hospital server; short-lived signing certificates (e.g. 1 year) renewed through the offline root
Pump → hospital session	Mutual authentication with device certificates (mutual TLS) or, for constrained devices, a per-device HMAC key (no non-repudiation needed); non-Kerberos because of clock/KDC dependence

Key and certificate management

- Issuance: the offline root CA (HSM, split control) signs a firmware-signing certificate (valid about 1 year). Device certificates and per-device keys are injected at manufacturing.
- Storage: signing private key in an HSM, never exported; root public key in pump ROM; version counter in protected non-volatile memory; device private key in a secure element.
- Rotation: the signing certificate is rotated yearly; an update can introduce a new signing certificate chained to the same root.
- Revocation: the signed revocation list is carried with each update and checked before verification; if the signing key is compromised, the root issues a new signing certificate, adds the old serial to the list and ships an emergency update signed by the new key.

Message flow for one update

Step	From → To	Content
1	Manufacturer build system → HSM	Image hash and manifest {model, version, size, SHA-256(image), issued} to be signed
2	HSM → Distribution server	Signature over the manifest; package = manifest + signature + signing certificate + revocation list + image
3	Server → Hospital → Pump	The package (any channel, untrusted); can be stored offline and delivered later
4	Pump	Runs the verification algorithm below; installs only if all checks pass
5	Pump → Hospital server	Reports installed version (optionally via mutual-TLS or HMAC-authenticated message)

Behaviour when offline or the clock is wrong

- Verification needs no network: the chain, signature and digest are checked locally with the pinned root key.
- The pump does not rely on its own clock for security decisions. Freshness comes from the version counter and the signed revocation list, not from timestamps. If a trusted time

(e.g. from the hospital server) is available, certificate validity is also enforced; otherwise the validity check is skipped (this is a deliberate trade-off and a residual risk).

- A pump that was offline simply accepts any newer valid version when it reconnects; it cannot be tricked into accepting an older one.

Pseudocode – signing (manufacturer)

```
SIGN_UPDATE(image, model, version):
  digest   <- SHA256(image)
  manifest <- {model, version, size = |image|, sha256 = digest, issued = now()}
  sig      <- ECDSA_Sign(signingKey_in_HSM, SHA256(manifest))   // nonce from HSM RNG / RFC 6979
  log(audit_trail, manifest, sig, operator, time)               // supports non-repudiation
  return package{manifest, sig, signingCert, revocationList, image}
```

Pseudocode – verification (pump)

```
VERIFY_UPDATE(pkg):
  cert <- pkg.signingCert
  if not ECDSA_Verify(rootPublicKey_in_ROM, cert.tbs, cert.signature):   REJECT
  if cert.serial in max(pkg.revocationList, storedRevocationList):       REJECT
  if trustedTime available and not (cert.notBefore <= trustedTime <= cert.notAfter): REJECT
  if not ECDSA_Verify(cert.publicKey, SHA256(pkg.manifest), pkg.sig):    REJECT
  m <- parse(pkg.manifest)
  if m.model != myModel:                                                 REJECT
  if m.version <= storedVersion:                                         REJECT   // rollback/replay
  if size(pkg.image) != m.size or SHA256(pkg.image) != m.sha256:         REJECT
  install(pkg.image); storedVersion <- m.version; save revocationList
  ACCEPT
```

Stage 4 – Validation Responsibilities

Student A – Security Argument Validation

Attack	What the attacker needs	Why the design stops it / effort
Modify the image	Same SHA-256 digest (second pre-image) or forge the manifest signature	Second pre-image ≈ 2^256 work; forging ECDSA P-256 ≈ 2^128 (Pollard rho)
Substitute a prepared pair of images (collision)	A chosen-prefix collision in SHA-256 and a signer who signs it	≈ 2^128 hash operations (Q1: effort 2^(n/2); Q1 measurement confirms the √ law). Nothing like the 2^16 – 2^32 effort of truncated toy hashes
Forge the signature	Break ECDSA or recover the signing key	≈ 2^128; Q3 shows the toy-key and nonce risks
Man-in-the-middle replaces file and digest	Signing key	Without the key, a replaced digest is not signed; the bare-digest baseline fails exactly here (see Student B)
Rollback to an old version	Genuine older signed package	Version counter rejects it (test T4)
Stolen signing key	Access to the HSM	Limited by HSM controls, short certificate lifetime, revocation list and emergency rotation (residual risk)
Compromised root	Root private key	Offline HSM, split control; would require re-provisioning pumps (severe

Attack	What the attacker needs	Why the design stops it / effort
		residual risk)
Wrong clock	Manipulate pump time	Security does not depend on the local clock; the version counter provides freshness
Repudiation by manufacturer	Claim the key was stolen	Audit trail, HSM logs, timestamps and revocation timeline defend the signature

Attacker effort estimates: digest SHA-256: collision ≈ 2^128, second pre-image ≈ 2^256 (Q1: n/2 rule and measured birthday growth); signature ECDSA P-256 ≈ 2^128 group operations (RSA-3072 would be about 2^128 as well); both are far beyond feasibility. Residual risks: compromise or insider misuse of the signing key, compromised root, implementation bugs (nonce reuse, parsing errors, TOCTOU between verification and flashing), side channels on pumps, and absence of trusted time for validity checks.

Student B – Experimental Validation

The final verification procedure was implemented in Python with ECDSA P-256, a root-signed signing certificate, a JSON manifest and a version counter. A baseline was a bare SHA-256 digest published beside the file.

```
T1 valid update (v6)                    -> ACCEPT | update installed | cost {'hash': 3, 'sigver': 2}
T2 modified image                       -> REJECT | image digest mismatch | cost {'hash': 3, 'sigver': 2}
T3 modified signature                   -> REJECT | manifest signature invalid | cost {'hash': 1, 'sigver': 1}
T4 replayed older version (v4 genuine)  -> REJECT | rollback/replay: version 4 <= installed 5 | cost {'hash': 2, 'sigver': 2}
```

T4b same version re-sent (v5) -> REJECT | rollback/replay: version 5 <= installed 5 | cost {'hash': 2, 'sigver': 2}
T5 expired signing cert (trusted time 2028) -> REJECT | signing certificate outside validity (trusted time) | cost {'hash': 1, 'sigver': 1}
T6 revoked signing cert (serial 1001 in list)-> REJECT | signing certificate revoked | cost {'hash': 1, 'sigver': 1}
T7 wrong key (attacker-signed manifest) -> REJECT | manifest signature invalid | cost {'hash': 1, 'sigver': 1}
T8 offline pump, wrong clock (no trusted time)-> ACCEPT | update installed | cost {'hash': 3, 'sigver': 2}
T9 wrong device model -> REJECT | wrong device model | cost {'hash': 2, 'sigver': 2}
added metadata bytes: manifest 150 sig 72 cert 334 total 556 (0.28% of 200 kB image)

Test	**Outcome**	**Reason**
T1 valid update (v6)	ACCEPT	All checks pass
T2 modified image	REJECT	Image digest mismatch
T3 modified signature	REJECT	Manifest signature invalid
T4 replayed older version / same version	REJECT	Rollback/replay check
T5 expired signing certificate (when trusted time exists)	REJECT	Validity check
T6 revoked signing certificate	REJECT	Revocation list
T7 wrong key (attacker-signed manifest)	REJECT	Signature invalid under the certificate key
T8 offline pump, no trusted time	ACCEPT	Valid update installs; expiry check skipped by design
T9 wrong device	REJECT	Model binding in manifest

Test	Outcome	Reason
model		

Comparison with the baseline (bare SHA-256 beside the file)

Baseline (bare SHA-256 beside file):
genuine file -> True
modified image, old digest -> False
attacker replaces file AND digest -> True (ACCEPTED - attack succeeds)
replayed older genuine version -> True (ACCEPTED - no version binding)
no signer identity / non-repudiation: none

Attack	Baseline (bare digest)	Final design
Modified image, old digest	Detected	Detected
Attacker replaces file and digest	NOT detected	Detected (signature)
Replay of older genuine version	NOT detected	Detected (version counter)
Wrong signer / unauthorised source	NOT detected	Detected (certificate chain and signature)
Revoked or compromised key	Not addressed	Handled by revocation list
Non-repudiation	None	Provided by signature and audit trail

Costs (measured)

Measured verification time on this PC: 0.43 ms per update (2 ECDSA verifies + 200 kB SHA-256)
SHA-256 of 200 kB alone: 0.141 ms

Metric	Value
Hash operations per update	3 (certificate TBS, manifest, image) – the image hash dominates, proportional to image size

Metric	Value
Signature verifications per update	2 (certificate by root, manifest by signing key)
Added metadata	556 bytes (manifest 150 + signature 72 + certificate 334), ≈ 0.28 % of a 200 kB image
Verification time on a PC	≈ 0.43 ms per update; on a microcontroller, ECDSA verify takes tens of milliseconds, still acceptable once per update

Stage 5 – Cross-Defence

Student A – what experimental evidence supports the design? Tests T2, T3, T4, T6, T7 and T9 show every modification, rollback, revocation, wrong-key and wrong-model case is rejected, while the genuine update is accepted (T1). The baseline accepted a replaced file with a recomputed digest and an older version, which the final design rejected. Q1 data supports digest-size choice (effort rises as 2^(k/2)), and Q3 data supports avoiding nonce reuse and textbook signatures.

Student B – how does the security argument follow from the mechanisms? SHA-256 in a signed manifest provides integrity (change the image and the digest changes; collisions cost 2^128). The ECDSA signature and certificate chain provide authentication, since only the key holder can sign and the chain binds the key to the manufacturer's root. The private key held by one party with an audit trail gives non-repudiation. The version counter gives freshness against rollback. The revocation list limits the effect of key compromise. Each mechanism closes a gap that the others leave open.

Stage 6 – Joint Evaluation

Topic	Evaluation
Integrity	Guaranteed by the SHA-256 digest inside the signed manifest
Authentication	Guaranteed by the ECDSA signature plus certificate chain to the pinned root

Topic	Evaluation
Non-repudiation	Provided by the signature with a single-holder (HSM) private key, plus audit trail/timestamps; limited by key-compromise claims
Freshness/rollback	Provided by the version counter, not by the clock
Costs	Verification time (two ECDSA verifications), about 0.5 kB per update, certificate storage and revocation-list handling, and key-management burden: an offline root, an HSM, rotation and audit procedures
Conditions under which it could still fail	Compromise of the signing key (until revoked), compromised root, implementation flaws (nonce reuse, parser bugs, non-atomic installation), no trusted clock so expired certificates go unnoticed offline, physical extraction of the pinned root key replaced by an attacker with hardware access
When a simpler/different mechanism fits	HMAC with a shared per-device key suits the pump–hospital session or a closed network where non-repudiation is not needed and devices are individually keyed; Kerberos tickets suit hospital internal single sign-on where clocks.